Lab Manual#3: Decision Trees

Prof. Dr. Aytug Onan

October 13, 2025

Contents

1 Introduction to Decision Trees **2**
1.1 Types of Decision Trees 2

2 Splitting Criteria **2**
2.1 Gini Impurity 2
2.2 Entropy 3
2.3 Mean Squared Error (MSE) 3

3 Implementing and Visualizing Decision Trees in Python **3**
3.1 Classification Tree using CART with Gini Impurity 3
3.2 Random Forests 4

4 Parameter Tuning for Decision Trees **6**
4.1 Hyperparameter Tuning with Grid Search 6

5 Project 1: Classification - Predicting Breast Cancer Diagnosis **6**
5.1 Objective 6
5.2 Dataset 6
5.3 Models Compared 7
5.4 Evaluation Metrics 7
5.5 Code Implementation 7
5.6 Expected Output 8

6 Project 2: Regression - Predicting House Prices **8**
6.1 Objective 10
6.2 Dataset 11
6.3 Models Compared 11
6.4 Evaluation Metrics 11
6.5 Code Implementation 11
6.6 Expected Output 12

1 Introduction to Decision Trees

Decision Trees are a type of supervised learning algorithm used for both classification and regression tasks. They work by splitting the data into subsets based on feature values, creating a tree-like structure that allows for decision-making based on those features.

1.1 Types of Decision Trees

Decision trees can vary based on their purpose and the metrics they use to create splits. Here are some common types:

- **CART (Classification and Regression Trees)**: Used in scikit-learn, this algorithm uses Gini impurity (for classification) or Mean Squared Error (for regression) to split data.
- **C4.5 and C5.0**: Extensions of the original ID3 algorithm, these algorithms use entropy and gain ratio to determine splits. They can handle both continuous and categorical features.
- **Random Forests**: An ensemble of decision trees trained on different subsets of the data. It averages or votes across trees to make decisions, improving accuracy and reducing overfitting.
- **Extra Trees (Extremely Randomized Trees)**: A variant of Random Forests where splits are chosen randomly, introducing more variance and further reducing overfitting.

2 Splitting Criteria

The criteria used to split data at each node of a decision tree directly impact the performance and interpretability of the model. We discuss three common criteria:

2.1 Gini Impurity

Gini impurity is used in classification tasks within CART decision trees to measure how "pure" each node is, or how mixed the classes are within the node.

Formula:

$$\text{Gini} = 1 - \sum_{i=1}^{C} p_i^2 \tag{1}$$

where:

- C is the total number of classes in the target variable.
- p_i is the probability of a sample belonging to class i in a given node.

A lower Gini value indicates a purer node, which is preferred as it leads to more homogenous splits.

2.2 Entropy

Entropy measures the impurity in a dataset. It is often used in C4.5 decision trees and is defined as:

Formula:

$$\text{Entropy} = -\sum_{i=1}^{C} p_i \log_2(p_i) \tag{2}$$

where:

- C is the number of classes.
- p_i is the proportion of samples in class i.

Entropy values range from 0 (pure node) to 1 (completely impure node), with lower entropy values preferred.

2.3 Mean Squared Error (MSE)

For regression trees, Mean Squared Error is used to split data based on continuous target values.

Formula:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{3}$$

where:

- N is the number of samples.
- y_i is the actual value of the i-th sample.
- $\hat{y}_i$ is the predicted value for the i-th sample.

3 Implementing and Visualizing Decision Trees in Python

We will now implement and visualize different types of decision trees using Python.

3.1 Classification Tree using CART with Gini Impurity

Here we use the CART algorithm with Gini impurity to train a classification decision tree.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load data and train model
```

```
iris = load_iris()
X, y = iris.data, iris.target
clf = DecisionTreeClassifier(criterion='gini', max_depth=3,
    random_state=42)
clf.fit(X, y)

# Visualize the tree
plt.figure(figsize=(12, 8))
plot_tree(clf, feature_names=iris.feature_names,
    class_names=iris.target_names, filled=True)
plt.show()
```

Expected Output: A tree visualization showing how the dataset splits based on Gini impurity. Each node displays Gini value, number of samples, and class distribution.

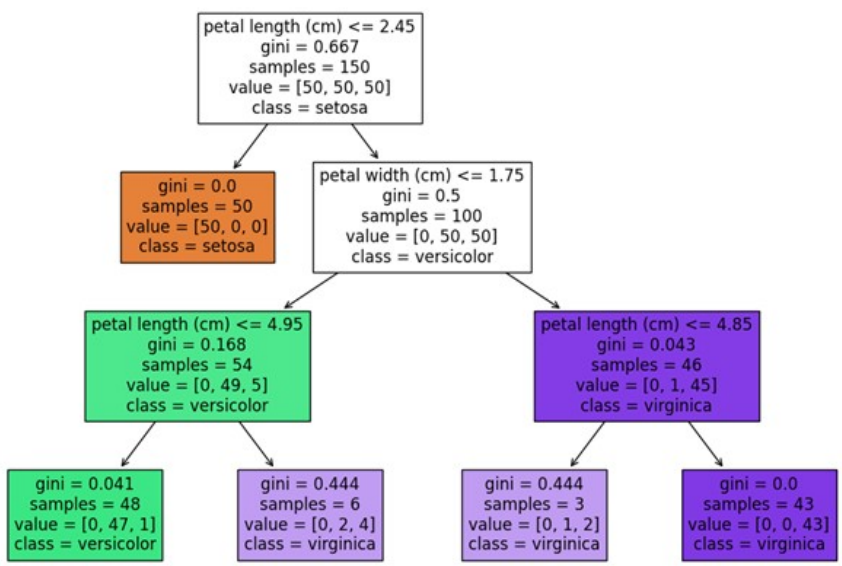

Figure 1: Tree visualization

3.2 Random Forests

Random Forests are an ensemble method where multiple decision trees are trained on random subsets of the data.

Code Implementation:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
```

```python
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Train Random Forest
rf = RandomForestClassifier(n_estimators=10, max_depth=3,
    random_state=42)
rf.fit(X, y)

# Visualize one of the trees in the forest
plt.figure(figsize=(12, 8))
plot_tree(rf.estimators_[0], feature_names=iris.feature_names,
    class_names=iris.target_names, filled=True)
plt.title("Visualization of One Tree in the Random Forest")
plt.show()
```

Expected Output: Visualization of one tree in the Random Forest, showing splits based on random subsets and features, with improved robustness against overfitting.

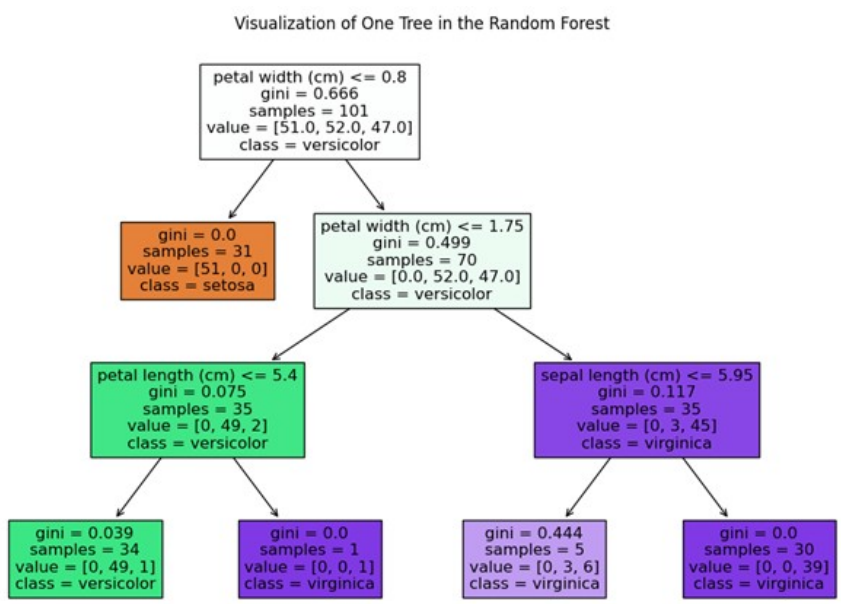

Figure 2: Tree Visualization

4 Parameter Tuning for Decision Trees

Key parameters that control tree complexity and performance include:

- **Max Depth**: Controls the maximum depth of the tree, affecting overfitting.
- **Min Samples Split**: Minimum samples required to split a node, controlling split sensitivity.
- **Min Samples Leaf**: Minimum number of samples in leaf nodes, ensuring broader splits.

4.1 Hyperparameter Tuning with Grid Search

Using Grid Search, we can find the optimal parameters for a decision tree.

```
from sklearn.model_selection import GridSearchCV

# Parameter grid
param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
    param_grid, cv=5)
grid_search.fit(X, y)

# Display best parameters
print("Best Parameters:", grid_search.best_params_)
```

5 Project 1: Classification - Predicting Breast Cancer Diagnosis

In this project, we use decision tree models to predict breast cancer diagnosis and compare their performance.

5.1 Objective

Use different decision tree-based models to classify breast cancer as malignant or benign, and compare their performance using accuracy and classification reports.

5.2 Dataset

The Breast Cancer dataset from scikit-learn, containing features like tumor size and texture, is used. The target labels are malignant and benign.

5.3 Models Compared

- Unpruned Decision Tree
- Pruned Decision Tree (limited depth)
- Random Forest Classifier
- Extra Trees Classifier

5.4 Evaluation Metrics

- Accuracy
- Precision, Recall, and F1 Score (for detailed class performance)

5.5 Code Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

# Define models
models = {
    "Unpruned␣Decision␣Tree": DecisionTreeClassifier(random_state=42),
    "Pruned␣Decision␣Tree": DecisionTreeClassifier(max_depth=5,
        random_state=42),
    "Random␣Forest": RandomForestClassifier(n_estimators=50,
        max_depth=5, random_state=42),
    "Extra␣Trees": ExtraTreesClassifier(n_estimators=50, max_depth=5,
        random_state=42)
}

# Train and evaluate models
train_accuracies, test_accuracies = [], []
for name, model in models.items():
    model.fit(X_train, y_train)
    train_accuracies.append(accuracy_score(y_train,
        model.predict(X_train)))
    test_accuracies.append(accuracy_score(y_test, model.predict(X_test)))
```

```
    print(f"\n{name} - Classification Report:\n",
        classification_report(y_test, model.predict(X_test)))

# Plot Training vs Test Accuracy
plt.figure(figsize=(10, 6))
x = range(len(models))
plt.bar(x, train_accuracies, width=0.4, label='Training Accuracy',
    align='center')
plt.bar([p + 0.4 for p in x], test_accuracies, width=0.4, label='Test
    Accuracy', align='center')
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.title('Training vs Test Accuracy for Breast Cancer Classification')
plt.xticks([p + 0.2 for p in x], list(models.keys()))
plt.legend()
plt.show()

# Plot Feature Importances
plt.figure(figsize=(12, 6))
for name, model in models.items():
    if hasattr(model, "feature_importances_"):
        plt.bar(range(len(model.feature_importances_)),
            model.feature_importances_, alpha=0.7, label=name)

plt.xlabel('Feature Index')
plt.ylabel('Importance')
plt.title("Feature Importances Across Models")
plt.legend()
plt.show()
```

5.6 Expected Output

- **Training vs Test Accuracy Plot**: Shows how well each model generalizes.
- **Feature Importances Plot**: Displays the most influential features for each model.
- **Classification Report**: Provides accuracy, precision, recall, and F1 scores.

6 Project 2: Regression - Predicting House Prices

In this project, we use decision tree models to predict housing prices and compare their performance.

```
Unpruned Decision Tree - Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.95      0.92        63
           1       0.97      0.94      0.95       108

    accuracy                           0.94       171
   macro avg       0.93      0.94      0.94       171
weighted avg       0.94      0.94      0.94       171


Pruned Decision Tree - Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        63
           1       0.96      0.96      0.96       108

    accuracy                           0.95       171
   macro avg       0.95      0.95      0.95       171
weighted avg       0.95      0.95      0.95       171


Random Forest - Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.94      0.95        63
           1       0.96      0.98      0.97       108

    accuracy                           0.96       171
   macro avg       0.97      0.96      0.96       171
weighted avg       0.96      0.96      0.96       171


Extra Trees - Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.92      0.95        63
           1       0.96      0.99      0.97       108

    accuracy                           0.96       171
   macro avg       0.97      0.96      0.96       171
weighted avg       0.97      0.96      0.96       171
```

Figure 3: Results

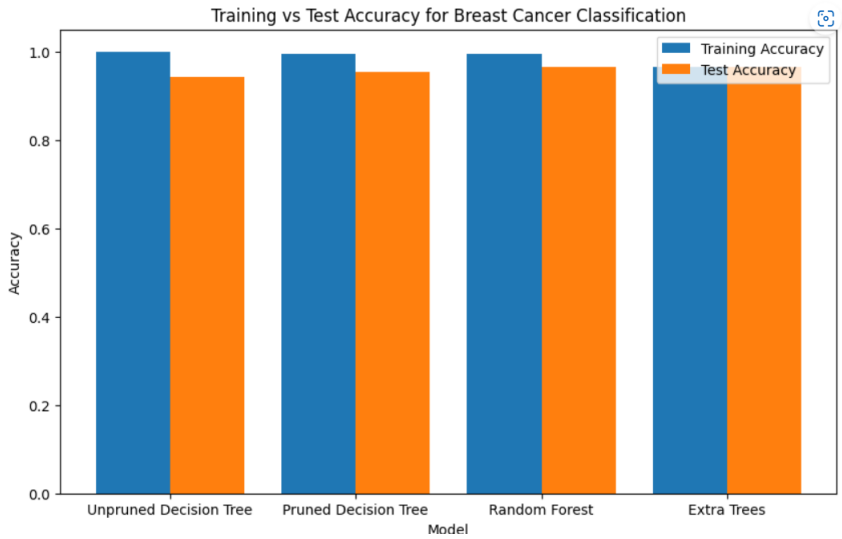

Figure 4: Histogram

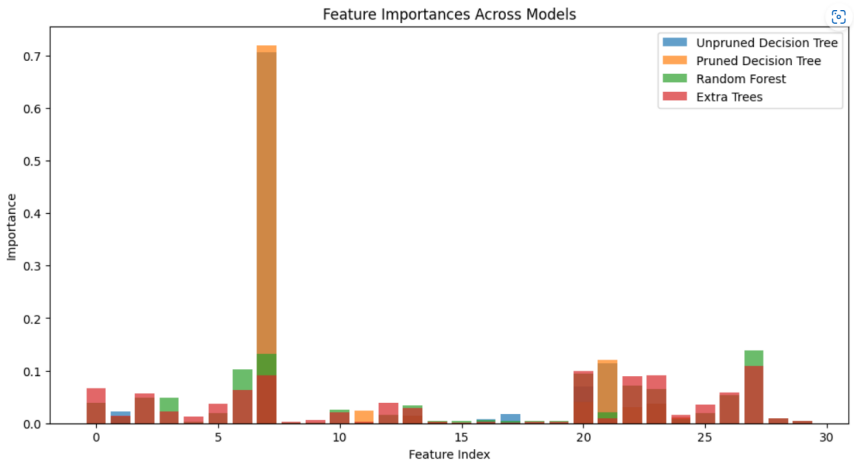

Figure 5: Feature Importance

6.1 Objective

Use decision tree-based models to predict California house prices, and evaluate performance using metrics like MSE, MAE, and R^2 score.

6.2 Dataset

The California Housing dataset from scikit-learn, containing features like median income and house age.

6.3 Models Compared

- Unpruned Decision Tree
- Pruned Decision Tree (limited depth)
- Random Forest Regressor
- Extra Trees Regressor

6.4 Evaluation Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R^2 Score

6.5 Code Implementation

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
import matplotlib.pyplot as plt

# Load dataset
data = fetch_california_housing()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

# Define models
models = {
    "Unpruned␣Decision␣Tree": DecisionTreeRegressor(random_state=42),
    "Pruned␣Decision␣Tree": DecisionTreeRegressor(max_depth=5,
        random_state=42),
    "Random␣Forest": RandomForestRegressor(n_estimators=50, max_depth=5,
        random_state=42),
    "Extra␣Trees": ExtraTreesRegressor(n_estimators=50, max_depth=5,
        random_state=42)
}
```

```python
# Train and evaluate models
train_mse, test_mse = [], []
for name, model in models.items():
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    train_mse.append(mean_squared_error(y_train, train_pred))
    test_mse.append(mean_squared_error(y_test, test_pred))
    print(f"\n{name} - Evaluation Metrics:")
    print("MAE:", mean_absolute_error(y_test, test_pred))
    print("MSE:", mean_squared_error(y_test, test_pred))
    print("R Score:", r2_score(y_test, test_pred))

# Plot Training vs Test MSE
plt.figure(figsize=(10, 6))
x = range(len(models))
plt.bar(x, train_mse, width=0.4, label='Training MSE', align='center')
plt.bar([p + 0.4 for p in x], test_mse, width=0.4, label='Test MSE',
     align='center')
plt.xlabel('Model')
plt.ylabel('Mean Squared Error')
plt.title('Training vs Test MSE for Housing Price Prediction')
plt.xticks([p + 0.2 for p in x], list(models.keys()))
plt.legend()
plt.show()

# Plot Feature Importances
plt.figure(figsize=(12, 6))
for name, model in models.items():
    if hasattr(model, "feature_importances_"):
        plt.bar(range(len(model.feature_importances_)),
            model.feature_importances_, alpha=0.7, label=name)

plt.xlabel('Feature Index')
plt.ylabel('Importance')
plt.title("Feature Importances Across Models")
plt.legend()
plt.show()
```

6.6 Expected Output

- **Training vs Test MSE Plot**: Visualizes how well each model generalizes.
- **Feature Importances Plot**: Highlights influential features in predicting house prices.
- **Regression Metrics**: Displays MAE, MSE, and R^2 scores.